```
# IMPORTANT: SOME KAGGLE DATA SOURCES ARE PRIVATE
# RUN THIS CELL IN ORDER TO IMPORT YOUR KAGGLE DATA SOURCES.
import kagglehub
kagglehub.login()
```

kaggle

Create an API token from your Kaggle settings page and paste it below.

API Token:

Login

Thank You

```
# IMPORTANT: RUN THIS CELL IN ORDER TO IMPORT YOUR KAGGLE DATA SOURCES,
# THEN FEEL FREE TO DELETE THIS CELL.
# NOTE: THIS NOTEBOOK ENVIRONMENT DIFFERS FROM KAGGLE'S PYTHON
# ENVIRONMENT SO THERE MAY BE MISSING LIBRARIES USED BY YOUR
# NOTEBOOK.

playground_series_s6e2_path = kagglehub.competition_download('playground-ser
print('Data source import complete.')
print(playground_series_s6e2_path)
```

```
Downloading to /root/.cache/kagglehub/competitions/playground-series-s6e2.arc
100%|██████████| 10.2M/10.2M [00:00<00:00, 146MB/s]Extracting files...

Data source import complete.
/root/.cache/kagglehub/competitions/playground-series-s6e2
```

1. Problem Definition

Cardiovascular disease is one of the leading causes of death worldwide. Early prediction of heart disease can help healthcare professionals identify high-risk patients and intervene earlier.

In this project, the goal is to build a machine learning model that predicts whether a patient is likely to develop heart disease based on clinical and demographic features.

Kaggle link : https://www.kaggle.com/competitions/playground-series-s6e2

```
# This Python 3 environment comes with many helpful analytics libraries inst
# It is defined by the kaggle/python Docker image: https://github.com/kaggle
# For example, here's several helpful packages to load

import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix, classification
import sklearn.metrics as metrics
```

2. Data Understanding

The dataset contains approximately 630,000 observations with 15 numerical features. The target variable indicates whether a patient has heart disease (1) or not (0).

Before building models, it is important to understand:

- the distribution of the target variable,
- relationships between features and the target,
- potential issues such as missing values or redundant features.

This step helps ensure that the modeling phase is based on meaningful and well-prepared data.

```
train = pd.read_csv("/root/.cache/kagglehub/competitions/playground-series-s
train.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 630000 entries, 0 to 629999
Data columns (total 15 columns):
 #   Column                   Non-Null Count   Dtype
---  ------                   --------------   -----
 0   id                       630000 non-null  int64
 1   Age                      630000 non-null  int64
 2   Sex                      630000 non-null  int64
 3   Chest pain type          630000 non-null  int64
 4   BP                       630000 non-null  int64
 5   Cholesterol              630000 non-null  int64
 6   FBS over 120             630000 non-null  int64
 7   EKG results              630000 non-null  int64
 8   Max HR                   630000 non-null  int64
 9   Exercise angina          630000 non-null  int64
 10  ST depression            630000 non-null  float64
 11  Slope of ST              630000 non-null  int64
 12  Number of vessels fluro  630000 non-null  int64
 13  Thallium                 630000 non-null  int64
 14  Heart Disease            630000 non-null  object
```

```
dtypes: float64(1), int64(13), object(1)
memory usage: 72.1+ MB
```

```
train.head(5)
```

	id	Age	Sex	Chest pain type	BP	Cholesterol	FBS over 120	EKG results	Max HR	Exercise angina	depres
0	0	58	1	4	152	239	0	0	158	1	
1	1	52	1	1	125	325	0	2	171	0	
2	2	56	0	2	160	188	0	2	151	0	
3	3	44	0	3	134	229	0	2	150	0	
4	4	58	1	4	140	234	0	2	125	1	

```
train.shape
```

```
(630000, 15)
```

There are 630000 rows in the training dataset, 15 features are all numerical.

3. Data Cleaning

In this step:

- the target column was encoded into a binary variable,
- unnecessary or redundant columns were removed,
- the dataset was checked for inconsistencies.

Clean and well-structured data is essential for building stable and interpretable machine-learning models.

```
train.describe()
```

	Age	Sex	Chest pain type	BP	Cholesterc
count	630000.000000	630000.000000	630000.000000	630000.000000	630000.00000
mean	54.136706	0.714735	3.312752	130.497433	245.01181
std	8.256301	0.451541	0.851615	14.975802	33.68158
min	29.000000	0.000000	1.000000	94.000000	126.00000
25%	48.000000	0.000000	3.000000	120.000000	223.00000
50%	54.000000	1.000000	4.000000	130.000000	243.00000
75%	60.000000	1.000000	4.000000	140.000000	269.00000
max	77.000000	1.000000	4.000000	200.000000	564.00000

```
DiseaseMatch = {"Presence":1, "Absence":0}
train["Disease"] = train["Heart Disease"].map(DiseaseMatch)
```

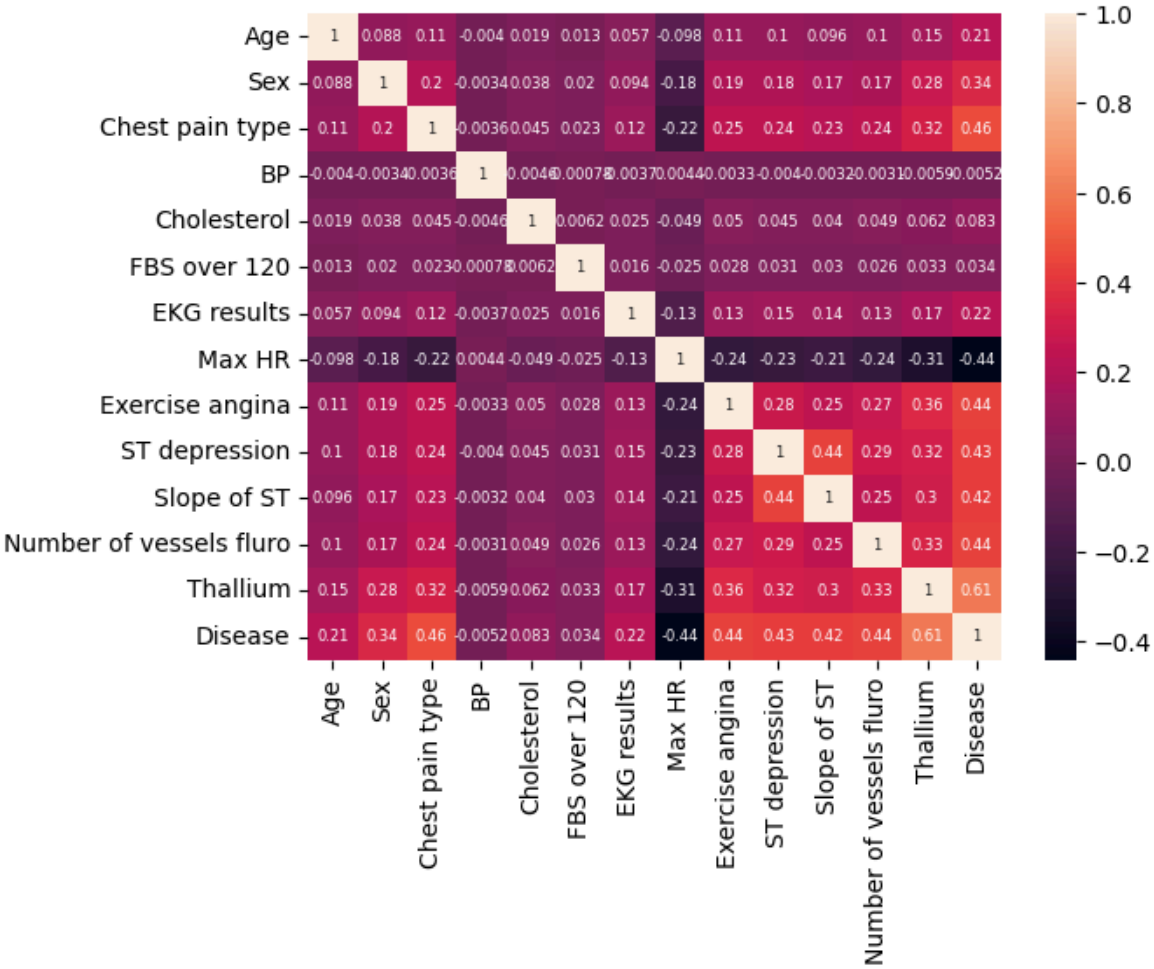
```
train = train.drop(columns = ["id", "Heart Disease"])
```

4. Exploratory Data Analysis (EDA)

```
import seaborn as sns
import matplotlib.pyplot as plt
```

```
sns.heatmap(train.corr(), annot = True, annot_kws = {"size":6})
```

```
<Axes: >
```



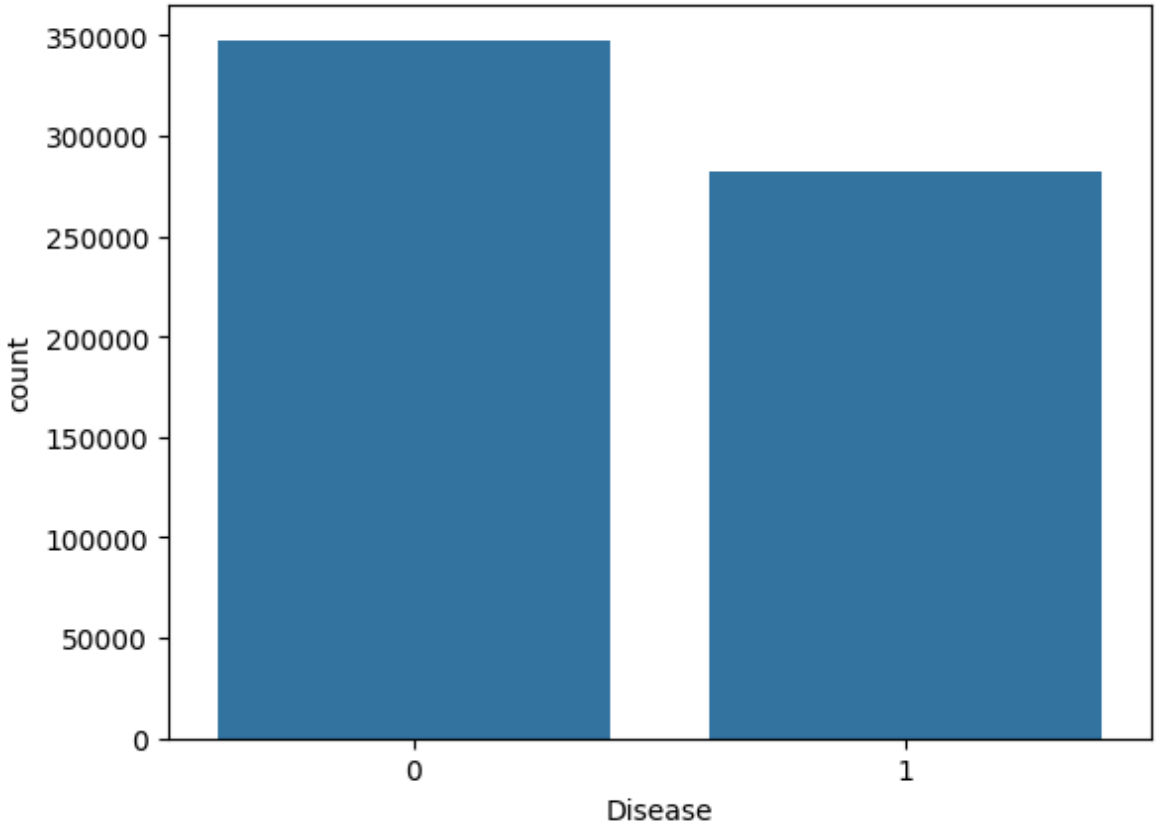
The purpose of EDA is to understand which features are most relevant for predicting heart disease.

Key findings:

1. Maximum Heart Rate (Max HR) has a negative relationship with getting heart disease.
2. Chest Pain type, Exercise angina, ST depression, Slope of ST and Thallium have a positive relationship with getting heart disease.
3. BP, Cholesterol, FBS over 120 show weak connection with getting heart disease.

```
sns.countplot(x = "Disease", data = train)
```

```
<Axes: xlabel='Disease', ylabel='count'>
```



There are around 350000 records for negative heart disease,

290000 records for positive heart disease.

This indicates that the dataset is relatively balanced, which is beneficial for training classification models and reduces the risk of bias toward one class.

5. Feature Engineering - Logistic regression

Logistic regression is used as a baseline model because:

1. it is simple and interpretable,
2. it allows us to understand the relationship between features and the probability of heart disease,
3. it provides a good benchmark before trying more complex models.

Regularization was applied to avoid overfitting and improve model generalization.

```
X = train.drop(columns = "Disease")
y = train["Disease"]

X_train, X_test, y_train, y_test = train_test_split(X,y, test_size = 0.2, ra
```

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train = pd.DataFrame(scaler.fit_transform(X_train), columns = X_train.colu
X_test = pd.DataFrame(scaler.fit_transform(X_test), columns = X_test.columns
```

6. Model Building - Logistic regression

Different values of the regularization parameter C were tested.

Findings:

- When C ≤ 5.7361e-06, the model is clearly underfitting because the penalty is too strong.
- The best performance is achieved when C ≈ 0.001, where the accuracy stabilizes around 0.882.

This shows that moderate regularization improves model performance while still keeping the model interpretable.

```
# Loop through a range of C values to decide which extent of penalty gives t
cs = np.logspace(-8,8,30)

for c in cs :
  model = LogisticRegression(
    max_iter=1000,
    random_state = 42,
    solver = "liblinear",
    penalty = "l1", # Using penalty to select the most useful features
    warm_start = True, # keep the coeficients from the previous training
    C = c)
  model.fit(X_train,y_train)
  y_pred = model.predict(X_test)
  accuracy = accuracy_score(y_test, y_pred)
  print(f"The accuracy of C {c} is {accuracy}")
```

```
The accuracy of C 1e-08 is 0.5520952380952381
The accuracy of C 3.562247890262444e-08 is 0.5520952380952381
The accuracy of C 1.2689610031679235e-07 is 0.5520952380952381
The accuracy of C 4.5203536563602405e-07 is 0.5520952380952381
The accuracy of C 1.6102620275609392e-06 is 0.5520952380952381
The accuracy of C 5.736152510448681e-06 is 0.5520952380952381
The accuracy of C 2.0433597178569398e-05 is 0.8513492063492063
The accuracy of C 7.278953843983146e-05 is 0.8783730158730159
The accuracy of C 0.0002592943797404667 is 0.8816587301587302
The accuracy of C 0.0009236708571873865 is 0.8821746031746032
The accuracy of C 0.003290344562312671 is 0.8821349206349206
The accuracy of C 0.011721022975334793 is 0.8821031746031746
The accuracy of C 0.041753189365604 is 0.8821111111111111
The accuracy of C 0.14873521072935117 is 0.8821031746031746
The accuracy of C 0.5298316906283702 is 0.882095238095238
The accuracy of C 1.8873918221350996 is 0.882095238095238
```

```
The accuracy of C 6.723357536499335 is 0.8820873015873016
The accuracy of C 23.95026619987481 is 0.8820873015873016
The accuracy of C 85.31678524172814 is 0.8820873015873016
The accuracy of C 303.9195382313195 is 0.8820873015873016
The accuracy of C 1082.6367338740563 is 0.8820873015873016
The accuracy of C 3856.620421163472 is 0.8820873015873016
The accuracy of C 13738.23795883261 is 0.8820873015873016
The accuracy of C 48939.00918477499 is 0.8820873015873016
The accuracy of C 174332.88221999872 is 0.8820873015873016
The accuracy of C 621016.9418915603 is 0.8820873015873016
The accuracy of C 2212216.29107045 is 0.8820873015873016
The accuracy of C 7880462.815669904 is 0.8820873015873016
The accuracy of C 28072162.039411698 is 0.8820873015873016
The accuracy of C 100000000.0 is 0.8820873015873016
```

```
model.classes_ # {"Presence":1, "Absence":0}
```

```
array([0, 1])
```

```
# Predict the probability of heart disease in the test dataset

test = pd.read_csv("/root/.cache/kagglehub/competitions/playground-series-s6
submission = pd.read_csv("/root/.cache/kagglehub/competitions/playground-ser
```

```
test = pd.DataFrame(scaler.fit_transform(test), columns = test.columns, inde

model = LogisticRegression(
    max_iter=1000,
    random_state = 42,
    solver = "liblinear",
    penalty = "l1", # Using penalty to select the most useful features
    warm_start = True, # keep the coeficients from the previous training
    C = 0.001)
model.fit(X_train,y_train)
test_prob = model.predict_proba(test)
```

```
Probs = []
for i in range(len(test_prob)):
  Probs.append(round(test_prob[i][1],3))
```

```
submission["Heart Disease"] = Probs
```

```
submission = submission.set_index("id")
```

```
submission.to_csv("Submission_heartDisease.csv")
```

7. Model Evaluation - Logistic regression

The logistic regression model was evaluated using:

- Accuracy
- ROC curve
- Classification report
- Log loss

The ROC curve shows that the model has good discriminative power, meaning it can distinguish between patients with and without heart disease relatively well.

Plot the ROC curve for logistic regression model.

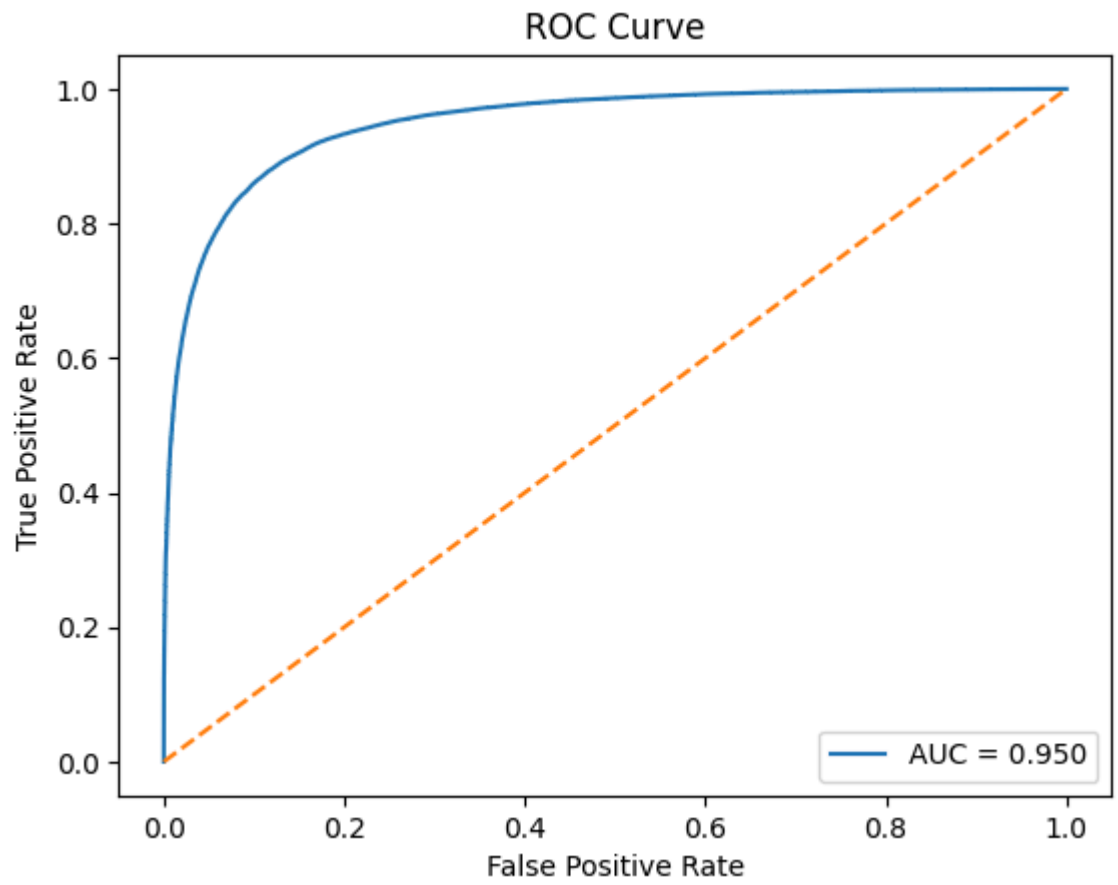
```
from sklearn.metrics import roc_curve, auc

y_prob = model.predict_proba(X_test)[:,1]
fpr, tpr, thresholds = roc_curve(y_test, y_prob)
roc_auc = auc(fpr, tpr)
print(f"AUC: {roc_auc:.4f}")
```

```
AUC: 0.9502
```

```
import matplotlib.pyplot as plt
plt.figure()
plt.plot(fpr,tpr, label = f"AUC = {roc_auc:.3f}")
plt.plot([0,1], [0,1], linestyle = "--")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.legend()

plt.show()
```



Classification report

```
from sklearn.metrics import classification_report
y_pred = model.predict(X_test)
print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       0.89      0.90      0.89     69564
           1       0.88      0.86      0.87     56436

    accuracy                           0.88    126000
   macro avg       0.88      0.88      0.88    126000
weighted avg       0.88      0.88      0.88    126000
```

Calculate log loss

```
loss = log_loss(y_test, y_prob)
print(f"Log Loss: {loss}")
```

```
Log Loss: 0.28290707300368567
```

8. Model Building - XGBoost

To improve performance, a more advanced model (XGBoost) was implemented.

XGBoost is a powerful gradient-boosting algorithm that is widely used in real-world machine-learning applications because:

- it handles large datasets efficiently,
- it captures non-linear relationships,
- it usually performs better than simple models such as logistic regression.

```
import xgboost as xgb

dtrain_clf = xgb.DMatrix(X_train, y_train, enable_categorical = True)
dtest_clf = xgb.DMatrix(X_test, y_test, enable_categorical = True)

params = {"objective": "binary:logistic", "tree_method": "hist"}
n = 1000

evals = [(dtrain_clf, "train"), (dtest_clf, "validation")]

model = xgb.train(
    params = params,
    dtrain = dtrain_clf,
    num_boost_round = n,
    early_stopping_rounds=50,
    evals = evals,
    verbose_eval = 20
)
```

```
[0]     train-logloss:0.52985   validation-logloss:0.54826
[20]    train-logloss:0.26967   validation-logloss:0.32491
[40]    train-logloss:0.26482   validation-logloss:0.32767
[60]    train-logloss:0.26224   validation-logloss:0.32881
[64]    train-logloss:0.26183   validation-logloss:0.32931
```

9. Model Evaluation - XGBoost

The XGBoost model was evaluated using the same metrics as the logistic regression model to ensure a fair comparison.

Cross-validation was also performed to check whether the model generalizes well to unseen data.

```
preds = model.predict(dtest_clf)
loss = log_loss(y_test, preds)
print(f"Log loss: {loss}")
```

```
Log loss: 0.3293073095916961
```

```
results = xgb.cv(
   params,
   dtrain_clf,
   num_boost_round=100,
   nfold=3,
   early_stopping_rounds=20,
   metrics = ["auc", "logloss"]
)
```

```
results.head()
```

	train-auc-mean	train-auc-std	train-logloss-mean	train-logloss-std	test-auc-mean	test-auc-std	test-logloss-mean	tes loglos s
0	0.939510	0.000212	0.529857	0.000232	0.939225	0.000575	0.529971	0.0001
1	0.944960	0.000252	0.443199	0.000371	0.944566	0.000389	0.443531	0.0002
2	0.946686	0.000327	0.389550	0.000400	0.946247	0.000245	0.390131	0.0004
3	0.948332	0.000287	0.354453	0.000467	0.947795	0.000455	0.355233	0.0006
4	0.949349	0.000084	0.330734	0.000442	0.948799	0.000645	0.331681	0.0008

Next steps: New interactive sheet

8. Conclusion

In this project, I built and evaluated multiple machine-learning models to predict the risk of heart disease using a large structured dataset with more than 600,000 observations.

The main results are:

- The logistic regression model achieved a log loss of 0.283, showing strong and stable performance.
- The XGBoost model achieved a log loss of 0.329, which indicates that the